

# AI Proctoring & Identity Verification Handover Report

## Full-Stack Architectural Implementation, Role Division, & Deployment Guide

|                                                                                                                   |                                                       |
|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Target Audience: Frontend & Backend Engineering Teams                                                             | Date: September 24, 2026                              |
| Database/Security Lead: Tanuj Sangwan                                                                             | Status: Completed, Tested & Deployed                  |
| Git Repository: <a href="https://github.com/no-usefun/quiz-app.git">https://github.com/no-usefun/quiz-app.git</a> | Active Branch: <a href="#">features/ai-proctoring</a> |
| Cloud Database: Neon PostgreSQL (neondb)                                                                          | Commit Hash: e0a2cc4                                  |

### 1. Executive Summary & System Architecture

This project delivers an enterprise-grade, real-time **AI Proctoring & Candidate Identity Verification System** designed to prevent academic dishonesty during online examinations. The system operates across three synchronized tiers:

|                                             |                                                                                                                                                                                                                                                                                              |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stage 1: Pre-Exam ID Verification           | Student webcam activates; UI displays an ID alignment guideline bounding box; candidate holds student ID card next to face; single-click high-res frame snapshot is captured to canvas and converted to base64 Data URL, then persisted in session storage and attached to the exam attempt. |
| Stage 2: Live Edge-AI Gaze & Audio Tracking | During the test, client-side computer vision tracks horizontal/vertical eye gaze quadrants and face presence (detecting 'Looking Away', 'No Face', 'Multiple Faces'). Concurrently, Web Audio API monitors ambient room volume for speech/noise spikes (>65dB).                              |
| Stage 3: Browser Lockdown & Auto-Submit     | Full-screen mode is strictly enforced with an inescapable lock overlay if exited. Tab switching, window blurring, right-clicking, and clipboard copying are intercepted in real-time. Reaching 3 cumulative integrity warnings automatically finalizes and submits the exam to Neon DB.      |

### 2. Engineering Roles & Responsibilities Breakdown

#### A. Database & Security Engineering (Tanuj Sangwan)

- **Neon PostgreSQL Migration:** Extended quiz\_attempts table with id\_photo\_data TEXT to store candidate ID verification photos.
- **Constraint Synchronization:** Updated PostgreSQL constraint activity\_logs\_activity\_type\_check to validate all 21 AI event types (LOOKING\_AWAY, FACE\_NOT\_DETECTED, MULTIPLE\_FACES, VOICE\_DETECTED, TAB\_SWITCH, FULLSCREEN\_EXIT, etc.).
- **Data Integrity & Cascade Rules:** Maintained foreign key indexes, cascade integrity, and zero-downtime compatibility with existing candidate attempts.

B. Backend Team Responsibilities (Spring Boot / Java 21)

- **Domain Entities:** Updated QuizAttempt.java (idPhotoData field) and ActivityType.java (violation flags).
- **Proctoring Service** (ProctoringService.java): Implemented warning counter incrementation, warning threshold evaluation, automatic submission triggers (AUTO\_SUBMITTED), and device telemetry logging (Device.java).
- **Candidate & Teacher REST APIs** (ProctoringController.java):
  1. POST /api/v1/attempts/{id}/activities — Logs real-time violations & returns live warning counts.
  2. POST /api/v1/attempts/{id}/id-photo — Uploads and stores candidate ID verification snapshot.
  3. POST /api/v1/attempts/{id}/device — Registers hardware/browser footprint.
  4. GET /api/v1/teacher/attempts/{id}/proctoring-summary — Delivers violation breakdown & ID photo to instructors.
- **Security Filter:** Configured SecurityConfig.java ensuring student and teacher role-based access control.

C. Frontend Team Responsibilities (Next.js 14 / TypeScript / Tailwind)

- **Verification Page** (src/app/test/[testCode]/verify/page.tsx): 2-step verification wizard with live camera viewfinder, visual card alignment guide, snapshot capture to canvas, preview/retake controls, and encrypted session storage.
- **AI Proctoring Hook** (src/hooks/useProctoring.ts): Client-side optical flow gaze tracker, face detection, Web Audio volume analyzer, tab switch listener (visibilitychange), fullscreen tracking, and copy-paste/right-click blockers.
- **Exam Arena HUD** (src/app/test/[testCode]/page.tsx): Picture-in-Picture live webcam feed, real-time AI status pill (Gaze Aligned / Looking Away / No Face), live warning counter (Warnings: X / 3), and non-dismissible fullscreen lock screen overlay.

3. Git Repository & Neon Cloud Deployment Status

| Component      | Target Environment & Coordinates                                          | Status              |
|----------------|---------------------------------------------------------------------------|---------------------|
| Git Repository | https://github.com/no-usefun/quiz-app.git                                 | PUSHED (UP TO DATE) |
| Git Branch     | features/ai-proctoring (Commit: e0a2cc4)                                  | MERGED & CLEAN      |
| Neon Cloud DB  | ep-wandering-frog-b3qd20n7-pooler.c-4.ap-southeast-1.aws.neon.tech/neondb | MIGRATED & LIVE     |
| Database DDL   | database/schema.sql & database/update_online_quiz_dev.sql                 | SYNCHRONIZED        |

## 4. Automated Verification & Test Results (100% Passed)

| Test Suite                                     | Scope / Details                                                                                                                                        | Result                    |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Neon Database Schema & Live Insert Test        | Tested live insert/select/delete of candidate ID snapshot and 11 distinct AI activity event types against Neon PostgreSQL.                             | 100% PASSED               |
| Spring Boot Proctoring Unit & Controller Tests | 14 automated tests in ProctoringControllerTest and ProctoringServiceTest verifying API status codes, auto-submit logic, and warning caps.              | 14/14 PASSED (0 failures) |
| Backend Compilation                            | Full project compile (98 source files) compiled cleanly with javac on Java 21 release target.                                                          | BUILD SUCCESS             |
| End-to-End Proctoring Lifecycle Simulation     | Complete workflow test: candidate ID scan -> attempt initialization -> 3 sequential violations -> auto-submission trigger -> instructor audit summary. | 100% PASSED               |

## 5. Engineering Handover Action Items

### For the Frontend Team:

1. Pull the latest branch: `git checkout features/ai-proctoring && git pull`.
2. Review the verified components in `src/app/test/[testCode]/verify/page.tsx` and `src/hooks/useProctoring.ts`.
3. Connect the Instructor Proctoring Summary page with GET `/api/v1/teacher/attempts/{id}/proctoring-summary` to render the candidate's captured ID photo snapshot alongside their exam timeline.

### For the Backend Team:

1. Confirm local environment points to the updated Neon Cloud connection string.
2. All schema changes and Java entities are already deployed and committed on `features/ai-proctoring`.
3. Proceed to merge `features/ai-proctoring` into `main` for release packaging.